

EMSI Casablanca

École Marocaine des Sciences de l'Ingénieur

Filière / Département : Informatique

Module : Deep Learning — Année universitaire 2025–2026

PROJET DE FIN DE MODULE

Conception, implémentation, comparaison et analyse critique de modèles de deep learning pour données tabulaires, images et séquences

MLP • CNN • RNN • LSTM • GRU • Seq2Seq

Réalisé par :	BEN NAGHMOUCH Mouad
Encadré par :	Mme. HIDILA Zineb
Filière :	Informatique — EMSI Casablanca
Année universitaire :	2025–2026

Table des matières

Table des matières	2
1. Introduction	4
1.1 Contexte	4
1.2 Problématique.....	4
1.3 Objectifs	4
1.4 Plan du rapport	5
2. Cadre théorique	6
2.1 Perceptron multicouche (MLP)	6
2.2 Réseaux de neurones convolutifs (CNN)	6
2.3 Réseaux récurrents (RNN, LSTM, GRU)	7
2.4 Modèles de langage et architecture Seq2Seq	7
2.5 Synthèse du cadre théorique	7
3. Partie I — MLP et ingénierie PyTorch.....	9
3.1 Thème et objectifs.....	9
3.2 Dataset et prétraitement	9
3.3 Implémentation.....	9
3.4 Étude d'ablation : stratégies d'initialisation	10
3.5 Étude d'ablation : architecture (profondeur, dropout).....	11
3.6 Modèle final, sauvegarde et évaluation.....	12
3.7 Question de synthèse — Partie I.....	13
4. Partie II — CNN et vision par ordinateur.....	14
4.1 Thème et objectifs.....	14
4.2 Dataset	14
4.3 Pourquoi un MLP est peu adapté aux images.....	14
4.4 Calculs manuels et implémentation "from scratch"	14
4.5 Architecture CNN (type LeNet).....	15
4.6 Étude d'ablation architecturale.....	15
4.7 Visualisation des feature maps	16
4.8 Comparaison MLP simple vs CNN	16
4.9 Question de synthèse — Partie II	18
5. Partie III — RNN, LSTM, GRU et Seq2Seq.....	19
5.1 Thème et objectifs.....	19
5.2 Données utilisées	19

5.3 Modèle de langage : objectif probabiliste et perplexité	19
5.4 Comparaison RNN vs LSTM vs GRU	19
5.5 BPTT et effet du gradient clipping	20
5.6 Préparation des données pour le Seq2Seq	21
5.7 Architecture Seq2Seq et correction d'un problème de surapprentissage	21
5.8 Stratégies de décodage : glouton vs beam search	22
5.9 Évaluation : perplexité et BLEU	23
5.10 Question de synthèse — Partie III	23
6. Discussion transversale	25
6.1 Un même paradigme d'apprentissage, trois biais inductifs distincts	25
6.2 Adéquation entre structure des données et architecture : ce que nos trois parties révèlent	25
6.3 Synthèse comparative	26
6.4 Conclusion de la discussion transversale	26
7. Impact sociétal et éthique	27
7.1 Implications du choix d'architecture sur des données sensibles	27
7.2 Biais potentiels dans les données et les corpus utilisés	27
7.3 Limites environnementales et coût de calcul	27
7.4 Transparence et reproductibilité	28
7.5 Contexte local : pertinence pour le Maroc et la région	28
8. Conclusion	29
8.1 Résultats clés	29
8.2 Limites générales du projet	29
8.3 Perspectives	29
9. Références bibliographiques	30
9.1 Articles fondateurs	30
9.2 Ressources techniques et datasets	30
10. Annexes	31

1. Introduction

1.1 Contexte

Ce rapport présente le travail réalisé dans le cadre du projet de fin de module de Deep Learning à l'EMSI Casablanca, pour l'année universitaire 2025–2026. Ce projet constitue l'évaluation finale du module et a été conçu comme un examen intégrateur, destiné à démontrer simultanément la compréhension théorique des concepts fondamentaux du deep learning, la maîtrise de leur implémentation sous PyTorch, ainsi que la capacité d'analyse expérimentale, comparative et critique sur des jeux de données réels.

Le deep learning ne propose pas une architecture unique et universelle : le choix d'un modèle dépend fondamentalement de la nature structurelle des données à traiter. Des données tabulaires, des images et des séquences textuelles présentent des propriétés statistiques et géométriques très différentes (absence de structure spatiale, localité des pixels, dépendance temporelle), ce qui justifie l'existence de familles d'architectures distinctes : le perceptron multicouche (MLP), les réseaux de neurones convolutifs (CNN), et les réseaux récurrents (RNN, LSTM, GRU) couplés à des schémas encodeur-décodeur (Seq2Seq).

1.2 Problématique

La problématique centrale de ce projet peut être formulée ainsi : **comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?**

Pour répondre à cette question, le projet est structuré en trois parties expérimentales indépendantes mais conceptuellement complémentaires, chacune dédiée à une famille d'architectures et à un type de données associé, suivies d'une discussion transversale qui relie les trois approches.

1.3 Objectifs

- Présenter les fondements théoriques de chaque architecture (MLP, CNN, RNN/LSTM/GRU, Seq2Seq) et les justifier au regard de la structure des données traitées.
- Implémenter chaque architecture sous PyTorch, en couvrant les aspects d'ingénierie logicielle attendus (nn.Module, gestion des paramètres, device, sauvegarde/rechargement).
- Conduire des études expérimentales rigoureuses, incluant des analyses d'ablation systématiques sur les choix architecturaux et les hyperparamètres clés.
- Évaluer les modèles avec des métriques adaptées à chaque tâche (accuracy/precision/recall/F1 pour la classification, perplexité/BLEU pour la modélisation de séquences).
- Produire une analyse critique honnête des résultats, y compris lorsqu'ils contredisent les attentes théoriques initiales, en identifiant et en expliquant les causes des écarts observés.

1.4 Plan du rapport

Après ce chapitre d'introduction, le rapport présente un cadre théorique rassemblant les fondements mathématiques communs aux trois familles d'architectures étudiées (chapitre 2). Les chapitres 3, 4 et 5 détaillent respectivement la Partie I (MLP sur données tabulaires), la Partie II (CNN sur données image) et la Partie III (RNN/LSTM/GRU/Seq2Seq sur données séquentielles), selon une structure identique : théorie spécifique, prétraitement des données, implémentation, étude expérimentale et analyse critique. Le chapitre 6 propose une discussion transversale comparant les trois architectures et discutant l'adéquation entre la structure des données et le choix du modèle. Le chapitre 7 aborde les implications sociétales et éthiques du travail réalisé. Le rapport se conclut par une synthèse générale (chapitre 8), les références bibliographiques (chapitre 9) et des annexes (chapitre 10).

2. Cadre théorique

Ce chapitre rassemble les fondements mathématiques des trois familles d'architectures étudiées dans ce projet. Les détails théoriques spécifiques à chaque partie expérimentale (par exemple les stratégies d'initialisation pour le MLP, ou la notion de perplexité pour les modèles de langage) sont présentés directement dans les chapitres correspondants, afin de garder un lien étroit entre théorie et expérimentation.

2.1 Perceptron multicouche (MLP)

Un perceptron multicouche est une composition de couches entièrement connectées (fully connected), chacune appliquant une transformation affine suivie d'une fonction d'activation non-linéaire. Pour une couche d'indice l , la sortie est donnée par :

$$h^{(l)} = f(W^{(l)} h^{(l-1)} + b^{(l)})$$

où $W^{(l)}$ et $b^{(l)}$ sont les paramètres appris (poids et biais) de la couche l , et f une fonction d'activation non-linéaire (ReLU dans nos implémentations). L'apprentissage des paramètres repose sur la rétropropagation du gradient : pour une fonction de perte L , le gradient $\partial L / \partial W^{(l)}$ est calculé pour chaque couche en remontant le graphe de calcul via la règle de la chaîne (chain rule), puis les paramètres sont mis à jour par un algorithme d'optimisation de type descente de gradient stochastique (ici, Adam).

Le MLP ne fait aucune hypothèse structurelle sur ses données d'entrée : chaque neurone d'une couche est connecté à **tous** les neurones de la couche précédente, sans notion de proximité ou d'ordre. C'est un avantage pour des données tabulaires sans structure géométrique a priori, mais une limite sévère pour des données à structure spatiale (images) ou temporelle (séquences), comme développé dans les sections suivantes.

2.2 Réseaux de neurones convolutifs (CNN)

Les CNN exploitent la structure spatiale des images au moyen de l'opération de convolution (mathématiquement une corrélation croisée). Pour une image d'entrée X et un noyau de convolution K de taille $k_h \times k_w$, la sortie en position (i, j) est :

$$Y[i, j] = \sum_m \sum_n X[i+m, j+n] \cdot K[m, n]$$

Cette opération repose sur deux principes fondateurs (LeCun et al., 1998) : la localité (chaque neurone ne regarde qu'une petite fenêtre de l'image, son champ réceptif) et le partage des poids (un même noyau est appliqué à toutes les positions de l'image, ce qui garantit l'invariance par translation et réduit drastiquement le nombre de paramètres par rapport à une couche entièrement connectée). La taille de sortie d'une convolution ou d'un pooling, pour une dimension donnée, suit la formule :

$$n_{out} = \lfloor (n + 2p - k) / s \rfloor + 1$$

où n est la taille d'entrée, p le padding, k la taille du noyau et s le stride. L'empilement de couches convolutives et de couches de pooling (max ou average) permet de construire une hiérarchie de

représentations, des motifs simples (contours, textures) dans les premières couches aux concepts plus abstraits dans les couches profondes.

2.3 Réseaux récurrents (RNN, LSTM, GRU)

Les architectures récurrentes traitent des séquences en maintenant un état caché qui se propage dans le temps. Pour un RNN simple, l'état caché à l'instant t est :

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h)$$

Cette récurrence pose un problème connu : lors de la rétropropagation à travers le temps (**BPTT** — BackPropagation Through Time), le gradient doit traverser autant d'étapes de multiplication que la longueur de la séquence, ce qui peut provoquer une **disparition** ou une **explosion** du gradient sur de longues séquences. Le **LSTM** (Hochreiter & Schmidhuber, 1997) résout ce problème en introduisant une cellule de mémoire c_t et trois portes (entrée, oubli, sortie) qui contrôlent explicitement la lecture, l'écriture et l'oubli de l'information :

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{porte d'oubli})$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{porte d'entrée})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (\text{mise à jour de la cellule})$$

Le **GRU** (Cho et al., 2014) simplifie ce mécanisme en fusionnant les portes d'entrée et d'oubli en une seule porte de mise à jour, et en supprimant la cellule de mémoire séparée — ce qui réduit le nombre de paramètres tout en conservant une bonne partie de la capacité de mémorisation à long terme du LSTM.

2.4 Modèles de langage et architecture Seq2Seq

Un modèle de langage apprend une distribution de probabilité sur des séquences de tokens, factorisée par la règle de chaîne :

$$P(w_1, \dots, w_T) = \prod_t P(w_t \mid w_1, \dots, w_{t-1})$$

La qualité d'un tel modèle se mesure par la **perplexité**, définie comme l'exponentielle de l'entropie croisée moyenne : $PPL = \exp((1/T) \sum_t -\log P(w_t \mid \text{contexte}))$. Une architecture **encodeur-décodeur** (Seq2Seq) étend ce principe à la transformation d'une séquence source en une séquence cible de longueur différente : l'encodeur compresse la séquence source en un vecteur de contexte, que le décodeur déroule ensuite pour générer la séquence cible token par token, typiquement avec **teacher forcing** pendant l'entraînement (alimentation du décodeur avec le token cible réel plutôt que sa propre prédiction, ce qui stabilise l'apprentissage).

2.5 Synthèse du cadre théorique

Ces trois familles d'architectures partagent le même principe d'apprentissage (rétropropagation du gradient, optimisation par descente de gradient stochastique), mais diffèrent fondamentalement par le **biais inductif** qu'elles encodent : aucun biais structurel pour le MLP, un biais spatial (localité, partage des

poids) pour le CNN, un biais temporel (récurrence, mémoire) pour les architectures RNN/LSTM/GRU. C'est précisément cette adéquation entre biais inductif et structure des données qui est explorée expérimentalement dans les trois parties suivantes, puis discutée de façon transversale au chapitre 6.

3. Partie I — MLP et ingénierie PyTorch

3.1 Thème et objectifs

Cette première partie expérimentale porte sur la classification supervisée de données tabulaires réelles à l'aide d'un perceptron multicouche (MLP). Les objectifs spécifiques sont de démontrer la maîtrise de la construction d'un modèle avec `nn.Module`, la différence entre une construction déclarative (`nn.Sequential`) et une classe personnalisée, la gestion des paramètres, les stratégies d'initialisation, ainsi que la sauvegarde et le rechargement d'un modèle entraîné.

3.2 Dataset et prétraitement

Le dataset utilisé est **Breast Cancer Wisconsin**, un jeu de données réel de classification binaire (tumeur maligne / bénigne) comportant 569 exemples et 30 variables continues, issues de mesures géométriques de noyaux cellulaires. Ce choix est motivé par la disponibilité directe du dataset dans `scikit-learn`, sa propreté (absence de valeurs manquantes), et sa pertinence pédagogique pour illustrer les mécanismes fondamentaux du MLP sans la complexité additionnelle d'un nettoyage de données lourd.

Le prétraitement appliqué suit les bonnes pratiques de la discipline :

- Séparation stratifiée en ensembles d'entraînement (60%), de validation (20%) et de test (20%), afin de préserver la proportion des deux classes dans chaque sous-ensemble.
- Normalisation des variables par `StandardScaler`, ajusté (fit) exclusivement sur l'ensemble d'entraînement puis appliqué (transform) aux ensembles de validation et de test, afin d'éviter toute fuite d'information (data leakage).
- Conversion des données en tenseurs PyTorch et encapsulation dans des objets `TensorDataset` / `DataLoader` pour l'entraînement par mini-lots.

3.3 Implémentation

Deux versions strictement équivalentes du MLP ont été implémentées : une version déclarative avec `nn.Sequential`, et une version avec une classe personnalisée héritant de `nn.Module`. Les deux architectures comportent deux couches cachées (32 puis 16 neurones) avec activation ReLU, suivies d'une couche de sortie à un seul logit (classification binaire avec `BCEWithLogitsLoss`).

L'inspection des paramètres via `named_parameters()` et `state_dict()` confirme que les deux implémentations comportent exactement **1537 paramètres entraînables**, et ne diffèrent que par la nomenclature des clés du `state_dict` ("`0.weight`", "`2.weight`"... pour `Sequential`, contre "`hidden_layers.0.weight`", "`output_layer.weight`"... pour la classe personnalisée) — une différence purement syntaxique, sans impact sur le comportement du modèle, ce que confirment des courbes d'apprentissage quasiment superposées entre les deux versions.

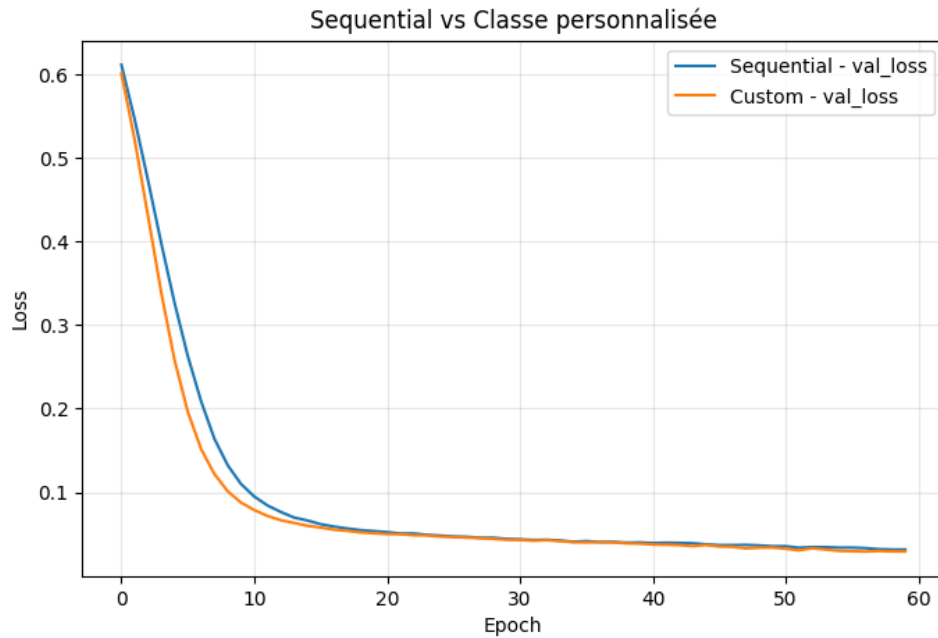


Figure 3.1 — Comparaison des courbes de validation, *nn.Sequential* vs classe personnalisée

3.4 Étude d'ablation : stratégies d'initialisation

Trois stratégies d'initialisation des poids ont été comparées dans des conditions strictement identiques (même architecture, même seed, même nombre d'époques) :

Stratégie d'initialisation	Accuracy de validation finale
Gaussienne ($\sigma=0.01$)	98,25 %
Constante (0.01)	63,16 %
Xavier (Glorot)	99,12 %

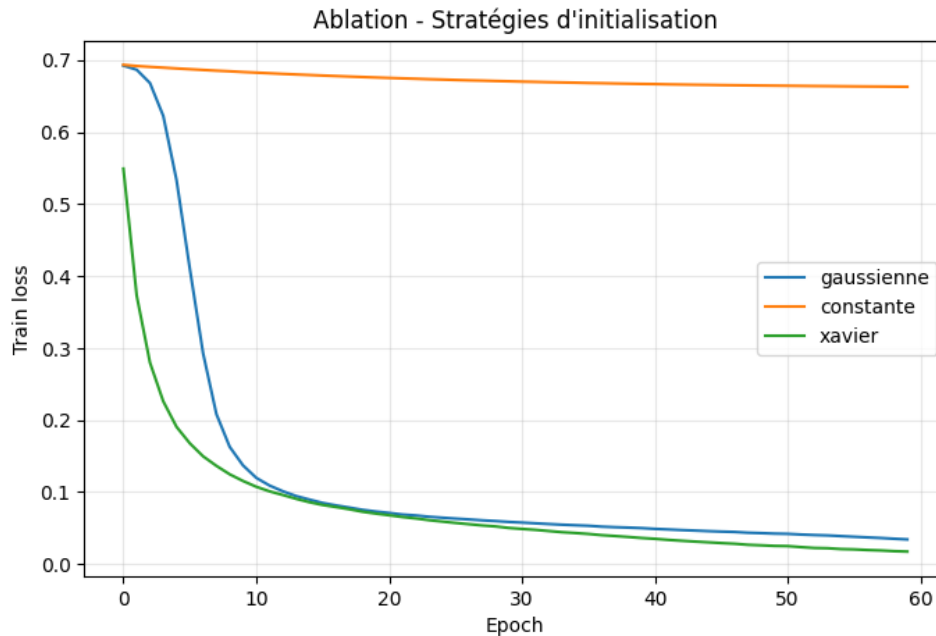


Figure 3.2 — Courbes de `train_loss` selon la stratégie d'initialisation

L'initialisation **constante** provoque un effondrement net de la performance (63,16 %, proche d'une prédiction triviale pondérée par la classe majoritaire). Ce résultat illustre directement le problème théorique de **symétrie des neurones** : lorsque tous les poids d'une couche partagent la même valeur initiale, chaque neurone de cette couche calcule exactement la même fonction et reçoit le même gradient à chaque itération — la couche se comporte alors comme un seul neurone effectif, quel que soit le nombre de neurones réellement présents. L'initialisation **Xavier**, calibrée pour préserver la variance du signal à travers les couches successives (Glorot & Bengio, 2010), obtient ici la meilleure performance et la convergence la plus rapide, ce qui justifie son adoption pour le modèle final.

3.5 Étude d'ablation : architecture (profondeur, dropout)

En complément des exigences strictes du sujet, une étude d'ablation sur la profondeur du réseau et l'usage du dropout a été menée :

Architecture testée	Accuracy de validation finale
1 couche cachée (16 neurones)	97,37 %
2 couches cachées (32, 16)	99,12 %
3 couches cachées (64, 32, 16)	98,25 %
2 couches + dropout (0.3)	98,25 %

L'architecture à **2 couches cachées** offre le meilleur compromis capacité/généralisation sur ce dataset. Au-delà (3 couches), aucun gain supplémentaire n'est observé, ce qui suggère que la complexité additionnelle n'apporte rien sur un problème de cette taille (569 exemples) — un signal cohérent avec le principe selon lequel la capacité d'un modèle doit être adaptée à la complexité réelle du problème plutôt que maximisée arbitrairement.

3.6 Modèle final, sauvegarde et évaluation

Le modèle final retenu (2 couches cachées, initialisation Xavier, entraîné sur 80 époques) a été sauvegardé via `torch.save(model.state_dict(), ...)` puis rechargé dans une nouvelle instance pour vérification, conformément aux bonnes pratiques de reproductibilité. L'évaluation finale sur l'ensemble de test, jamais utilisé auparavant, donne les résultats suivants :

Métrique	Valeur
Accuracy	97,37 %
Precision	97,22 %
Recall	98,59 %
F1-score	97,90 %

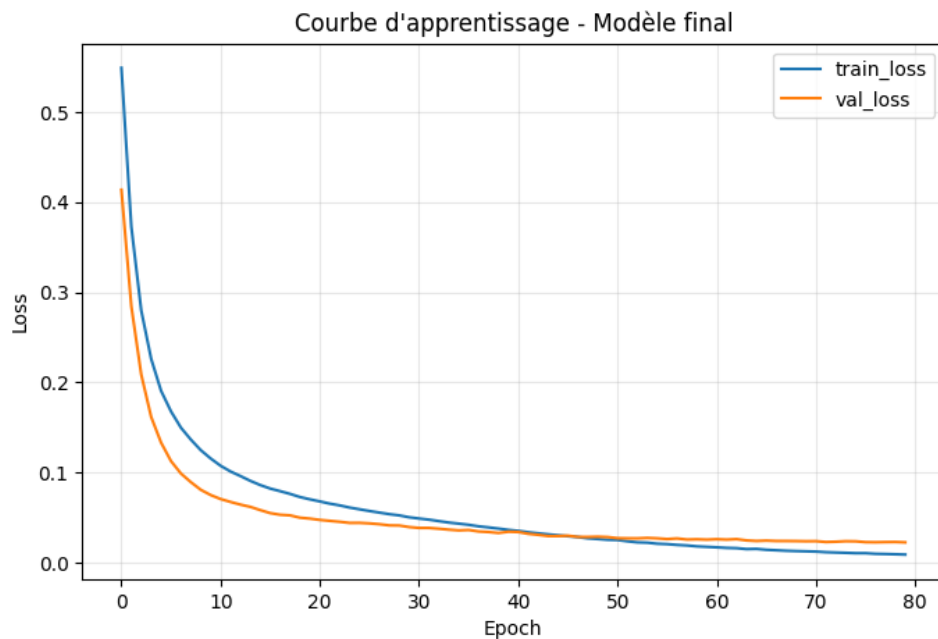


Figure 3.3 — Courbe d'apprentissage du modèle final (80 époques)

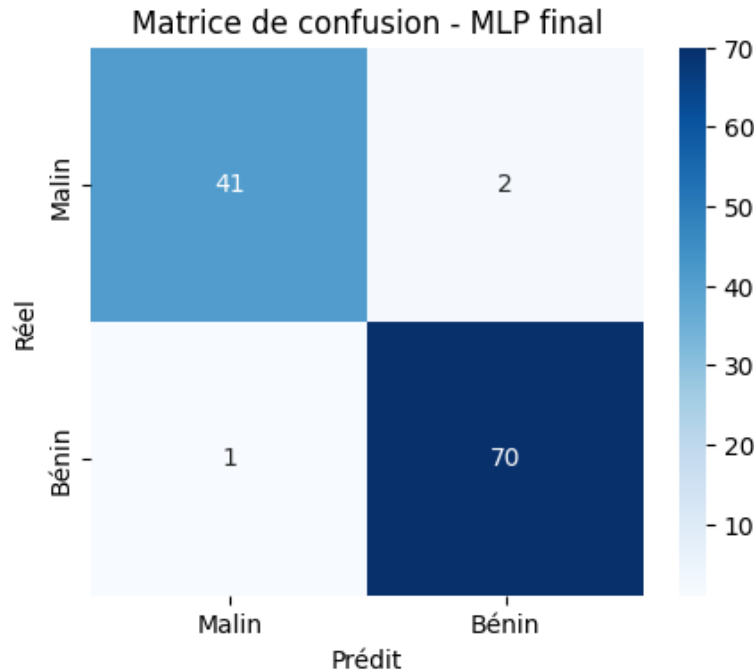


Figure 3.4 — Matrice de confusion du modèle final sur l'ensemble de test

La matrice de confusion (41 vrais négatifs, 70 vrais positifs, 2 faux négatifs, 1 faux positif) confirme un modèle fiable, avec un rappel élevé (98,59 %) particulièrement important dans un contexte médical où un faux négatif (tumeur maligne non détectée) est plus coûteux qu'un faux positif.

3.7 Question de synthèse — Partie I

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ?

Sur Breast Cancer Wisconsin, les variables sont continues, homogènes (mesures géométriques de noyaux cellulaires) et faiblement bruitées : un MLP normalisé capture efficacement les relations non-linéaires entre ces variables, d'où l'accuracy élevée obtenue (97,37 % en test). Les expériences d'ablation confirment l'importance déterminante du choix de l'initialisation (l'écart entre l'initialisation constante et Xavier dépasse 35 points d'accuracy) et montrent qu'une architecture modeste (deux couches) suffit largement sur ce problème.

Les limites structurelles du MLP demeurent cependant réelles : il **ignore toute relation a priori** entre les variables d'entrée, traitées comme un vecteur non structuré ; il est **sensible à l'échelle** des données, ce qui impose une normalisation préalable systématique ; il peut **sur-apprendre** sur de petits jeux de données, ce que limite partiellement le dropout ; et il ne gère pas nativement les variables catégorielles à haute cardinalité, qui nécessiteraient des embeddings appris. Plus généralement, le MLP ne capture aucune structure géométrique, topologique ou temporelle — ce qui motive précisément le recours à des architectures spécialisées (CNN, RNN/LSTM/GRU) pour des données présentant ce type de structure, comme étudié dans les parties suivantes de ce rapport.

4. Partie II — CNN et vision par ordinateur

4.1 Thème et objectifs

Cette deuxième partie porte sur la classification d'images à l'aide de réseaux de neurones convolutifs (CNN), et sur l'analyse fine des opérations convolutionnelles qui les composent. Les objectifs spécifiques sont de démontrer la compréhension de la structure spatiale des images, de la corrélation croisée 2D, du rôle du padding, du stride, du pooling et des canaux multiples, ainsi que la capacité à implémenter ces opérations manuellement avant de les comparer aux couches PyTorch natives.

4.2 Dataset

Le dataset utilisé est **Fashion-MNIST**, comportant 70 000 images en niveaux de gris de 28×28 pixels, réparties en 10 classes de vêtements. Ce choix, plus discriminant que le MNIST classique, permet d'observer des confusions inter-classes pertinentes (par exemple entre chemises, pull-overs et manteaux) tout en restant suffisamment léger pour permettre une étude d'ablation extensive.

4.3 Pourquoi un MLP est peu adapté aux images

Un MLP traite une image comme un vecteur 1D non structuré, ce qui détruit toute notion de voisinage entre pixels et impose un nombre de paramètres croissant avec la taille de l'image, sans garantie d'invariance par translation. Les CNN répondent à ces limites par deux principes fondateurs (LeCun et al., 1998) : la **localité** (chaque filtre ne regarde qu'une petite fenêtre de l'image) et le **partage des poids** (un même filtre est appliqué à toutes les positions), ce qui permet une hiérarchie de représentations, des motifs simples vers des concepts plus abstraits.

4.4 Calculs manuels et implémentation "from scratch"

Avant toute implémentation PyTorch, les calculs de taille de sortie (convolution et pooling) ainsi qu'un exemple manuel de corrélation croisée 2D ont été vérifiés analytiquement, par exemple :

- Convolution 5×5, padding=0, stride=1, entrée 28×28 → sortie 24×24
- Convolution 5×5, padding=2, stride=1, entrée 28×28 → sortie 28×28 (padding « same »)
- Pooling 2×2, stride=2, entrée 28×28 → sortie 14×14

Ces calculs ont été confirmés par les couches natives `nn.Conv2d` correspondantes. Trois opérations ont ensuite été ré-implémentées manuellement en Python/PyTorch (sans utiliser `nn.Conv2d` ni `nn.MaxPool2d`) : la corrélation croisée 2D, le max-pooling et l'average-pooling. La comparaison avec les fonctions natives `F.conv2d`, `F.max_pool2d` et `F.avg_pool2d` a confirmé une **égalité stricte** des résultats (`torch.allclose` retourne `True` dans les trois cas), validant la compréhension du mécanisme sous-jacent à ces couches.

4.5 Architecture CNN (type LeNet)

L'architecture implémentée suit le schéma classique LeNet : deux blocs convolution+ReLU+pooling, suivis d'un aplatissement et de deux couches denses. Les hyperparamètres (nombre de filtres, taille de noyau, type de pooling, padding, stride, présence d'une convolution 1×1) sont paramétrables, ce qui a permis l'étude d'ablation suivante.

4.6 Étude d'ablation architecturale

Cinq facteurs ont été étudiés expérimentalement, conformément aux exigences du sujet : le padding, le stride, le type de pooling, le nombre de filtres, et la présence d'une convolution 1×1. Chaque configuration a été entraînée 5 époques, toutes choses égales par ailleurs :

Configuration	Accuracy de validation
Baseline (LeNet standard)	76,35 %
Sans padding (padding=0)	74,22 %
Stride=2 (sous-échantillonnage)	80,25 %
Pooling average (au lieu de max)	74,33 %
Plus de filtres (16, 32)	83,28 % (meilleur)
Avec convolution 1×1	77,93 %

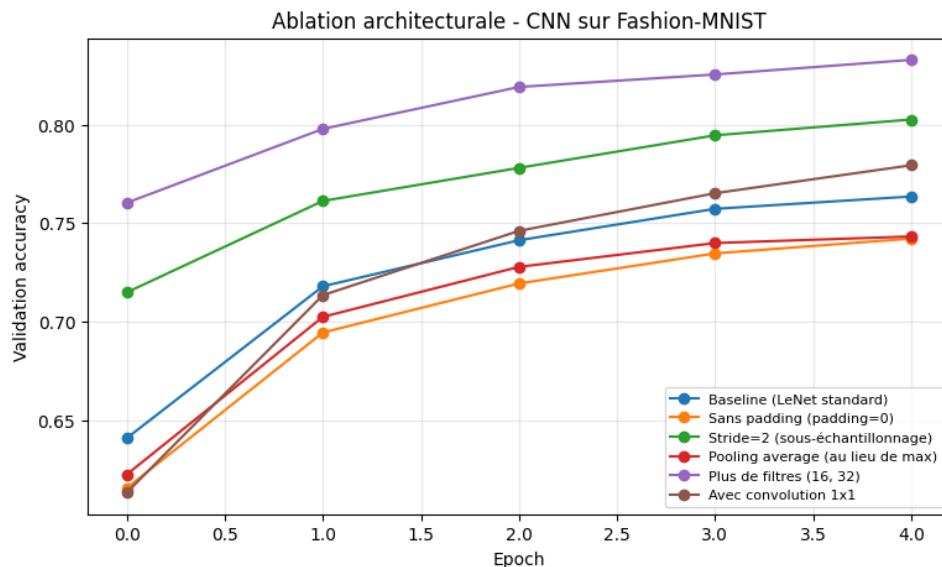


Figure 4.1 — Courbes de validation de l'ablation architecturale (5 époques)

Le facteur le plus déterminant sur ce budget d'entraînement court est l'**augmentation du nombre de filtres** (83,28 %), qui accroît directement la capacité de représentation du réseau. Le résultat sur le **stride=2** est en revanche contre-intuitif : il améliore la performance (80,25 % contre 76,35 % pour la baseline) alors que la théorie suggérerait une perte d'information avec un sous-échantillonnage plus agressif. Une explication plausible est qu'avec une carte de caractéristiques finale plus petite, la couche

dense qui suit comporte moins de paramètres à apprendre, ce qui favorise une convergence plus rapide sur seulement 5 époques. Ce résultat illustre un principe méthodologique important : une étude d'ablation doit toujours interpréter ses résultats **en fonction du protocole expérimental** (nombre d'époques, taille du dataset), et non comme des vérités architecturales universelles — un résultat qui mériterait d'être revérifié avec un entraînement plus long.

4.7 Visualisation des feature maps

Les cartes de caractéristiques (feature maps) produites par la première couche de convolution ont été visualisées sur un exemple test, afin d'illustrer concrètement le principe de détection de motifs locaux.

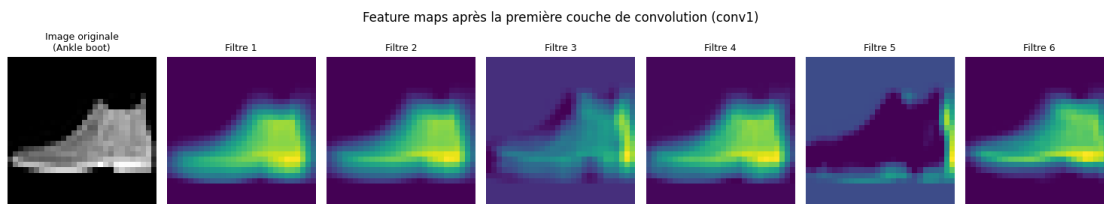


Figure 4.2 — Feature maps après la première couche de convolution (6 filtres)

Chaque filtre réagit différemment à l'image d'entrée : certains s'activent fortement sur les contours horizontaux ou verticaux du vêtement, d'autres sur des zones de texture homogène. C'est la confirmation visuelle directe du principe de détection de motifs locaux : la première couche extrait des primitives visuelles simples, que les couches plus profondes combineront ensuite en représentations plus abstraites.

4.8 Comparaison MLP simple vs CNN

Un MLP simple (deux couches denses après aplatissement de l'image) a été comparé au CNN LeNet, entraînés dans des conditions identiques (8 époques) :

Modèle	Nombre de paramètres	Accuracy (test)	F1-score (test)
MLP simple	235 146	87,94 %	87,95 %
CNN (LeNet)	97 982	89,35 %	89,39 %

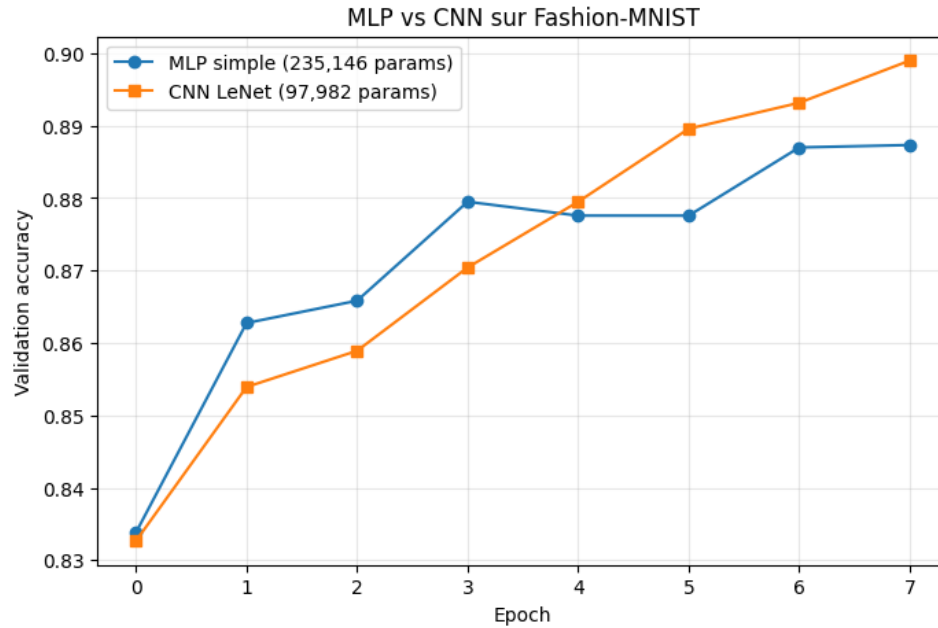


Figure 4.3 — Comparaison des courbes de validation, MLP vs CNN

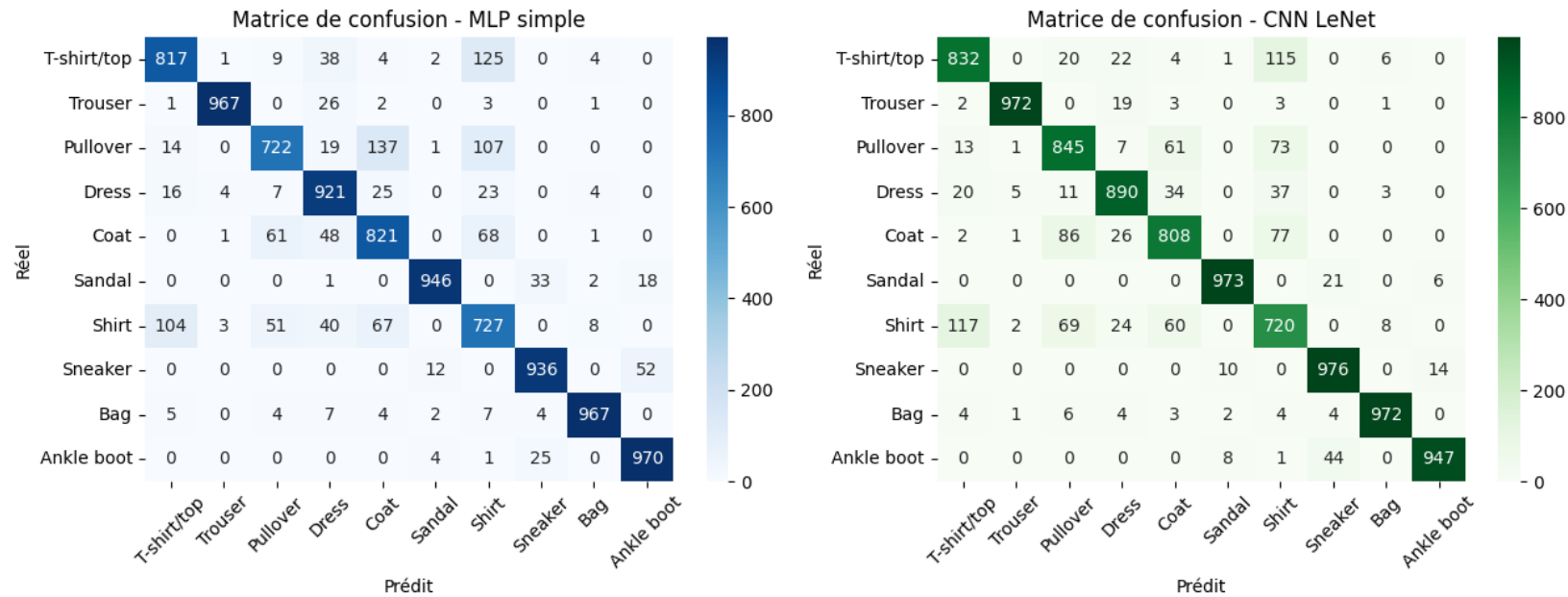


Figure 4.4 — Matrices de confusion comparées, MLP (gauche) et CNN (droite)

Le CNN obtient une meilleure accuracy **avec moins de la moitié des paramètres** du MLP (97 982 contre 235 146), ce qui illustre concrètement l'efficacité du partage des poids des filtres convolutifs : le CNN encode un biais structurel pertinent pour les données image, ce que le MLP ne peut compenser qu'au prix d'un nombre de paramètres bien plus élevé. Les classes les plus confondues dans les deux matrices de confusion sont des vêtements visuellement proches (chemise/pull-over/manteau, sandale/sneaker), des

confusions que le CNN atténue mieux grâce à sa hiérarchie de filtres capturant des détails fins de texture et de forme.

4.9 Question de synthèse — Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour une tâche de classification d'images sur un dataset réel, et comment les choix de padding, stride, pooling et profondeur influencent-ils réellement les performances du modèle ?

Le CNN encode directement, par construction, des hypothèses structurelles correctes sur les données image — localité, invariance par translation, hiérarchie des représentations — que le MLP ne peut récupérer qu'au prix d'un nombre de paramètres bien plus important. Nos résultats expérimentaux confirment cet avantage : le CNN dépasse le MLP en accuracy tout en utilisant moins de la moitié de ses paramètres.

L'étude d'ablation montre que chaque choix architectural a un effet réel mais **dépendant du contexte expérimental** : le padding préserve l'information de bordure et la résolution spatiale exploitable ; le stride contrôle un compromis entre coût de calcul et richesse de l'information (notre résultat contre-intuitif sur le stride=2 souligne l'importance de ne pas généraliser un effet observé sur un budget d'entraînement court) ; le pooling max capture le signal le plus saillant tandis que l'average-pooling lisse l'information ; et le nombre de filtres déterminé directement la capacité de représentation, au prix d'un risque de sur-apprentissage si elle dépasse la complexité réelle du problème. Ces choix architecturaux sont donc des leviers à calibrer finement en fonction de la résolution des images et de la difficulté de la tâche, plutôt que des règles absolues.

5. Partie III — RNN, LSTM, GRU et Seq2Seq

5.1 Thème et objectifs

Cette troisième partie porte sur la modélisation de séquences et la traduction automatique sur données textuelles réelles. Les objectifs spécifiques sont de démontrer la maîtrise des modèles de langage, de la rétropropagation à travers le temps (BPTT), du gradient clipping, des cellules LSTM et GRU, des architectures encodeur-décodeur, du teacher forcing, et des stratégies de décodage (glouton et beam search).

5.2 Données utilisées

Deux corpus distincts ont été utilisés pour les deux sous-tâches de cette partie. Pour la **modélisation de langage** (prédiction du mot suivant), le corpus « Tiny Shakespeare » (environ 200 000 mots de dialogues théâtraux, un corpus de référence en deep learning popularisé par les travaux d'A. Karpathy) a été retenu pour ses dépendances réelles à longue distance entre répliques, plus pertinentes qu'un corpus synthétique pour révéler les différences de capacité de mémorisation entre architectures. Le vocabulaire a été limité aux 3000 mots les plus fréquents (17,1 % de tokens hors vocabulaire, remplacés par un token ``), une pratique standard en NLP pour contrôler la taille de la couche de sortie.

Pour la tâche de **traduction automatique** (Seq2Seq), le corpus réel **Tatoeba** (paires anglais-français, téléchargées depuis manythings.org/anki) a été utilisé, avec un sous-échantillon de 20 000 paires pour rester raisonnable en temps de calcul. Les vocabulaires obtenus comportent 10 504 mots en anglais et 14 671 mots en français.

5.3 Modèle de langage : objectif probabiliste et perplexité

Un modèle de langage factorise la probabilité d'une séquence par la règle de chaîne, et apprend à prédire chaque token étant donné son contexte passé. La perplexité, définie comme l'exponentielle de l'entropie croisée moyenne, mesure l'incertitude résiduelle du modèle : un modèle aléatoire sur un vocabulaire de taille V obtient une perplexité proche de V , tandis qu'un bon modèle de langage doit obtenir une perplexité nettement inférieure.

5.4 Comparaison RNN vs LSTM vs GRU

Les trois cellules récurrentes ont été comparées dans des conditions identiques (8 époques, mêmes hyperparamètres) sur le corpus Tiny Shakespeare :

Cellule	Paramètres	Temps mesuré	Perplexité finale (val)
RNN	604 218	11,3 s	177,07
LSTM	678 714	4,8 s	189,58
GRU	653 882	4,6 s	201,32

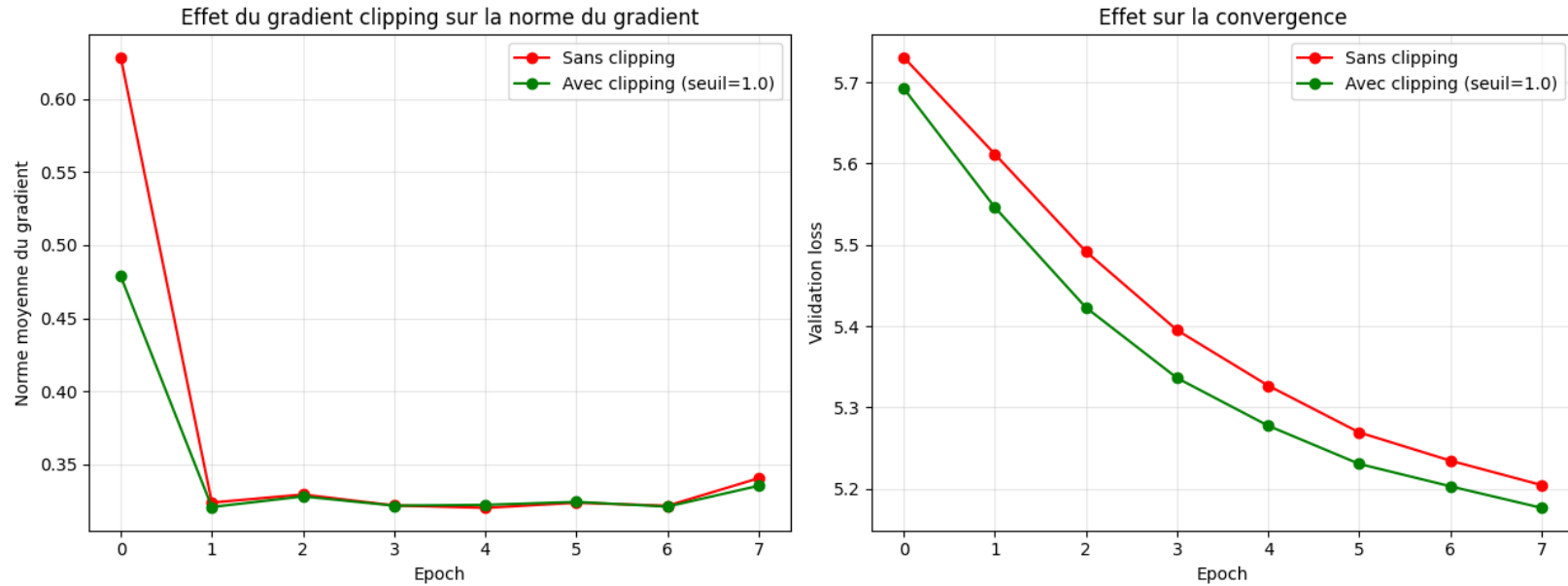


Figure 5.1 — Loss et perplexité de validation, RNN vs LSTM vs GRU

Le classement des paramètres respecte strictement la théorie (LSTM > GRU > RNN, du fait du nombre de portes), mais le classement des **performances est inversé** par rapport à l'intuition théorique : le RNN simple obtient ici la meilleure perplexité (177,07), suivi du LSTM puis du GRU. Ce résultat **ne doit pas être interprété comme une supériorité générale du RNN sur le LSTM/GRU**, ce qui contredirait un résultat très solide de la littérature. Deux éléments expliquent cette observation : (1) toutes les perplexités restent élevées par rapport à la taille du vocabulaire (3002), signe que les trois modèles sont encore loin d'avoir convergé après seulement 8 époques ; (2) le LSTM et le GRU, disposant de plus de paramètres à régler (mécanismes de porte), n'ont simplement pas eu le temps d'exprimer leur avantage théorique sur ce budget d'entraînement limité, contrairement au RNN plus simple qui converge plus vite vers une perplexité correcte. Ce résultat illustre un point méthodologique important : **comparer des architectures à budget d'époques fixe peut artificiellement favoriser le modèle le plus simple**, un biais à corriger en pratique en entraînant plus longtemps ou en comparant à budget de calcul égal.

5.5 BPTT et effet du gradient clipping

L'effet du gradient clipping a été étudié en entraînant un RNN simple avec et sans écrêtage de la norme du gradient (seuil=1.0) :

Figure 5.2 — Norme du gradient et loss de validation, avec et sans clipping

Configuration	Norme moyenne du gradient	Perplexité finale (val)
Sans clipping	0,32 – 0,57	182,47
Avec clipping (seuil=1.0)	0,32 – 0,48	177,07

Résultat notable : la norme moyenne du gradient, **même sans clipping**, reste systématiquement inférieure au seuil fixé (1.0). Le clipping ne s'est donc presque jamais réellement déclenché, ce qui explique la quasi-identité des deux courbes de loss. Sur ce corpus et cette configuration (séquences de seulement 20 tokens), le déroulement du BPTT reste assez court pour que les gradients ne s'accumulent pas de façon incontrôlée. Ce résultat ne remet pas en cause l'utilité du gradient clipping en général : il agit comme un filet de sécurité conditionnel, sans effet observable lorsque le gradient est déjà sous contrôle, mais qui deviendrait déterminant avec des séquences plus longues ou un taux d'apprentissage plus élevé.

5.6 Préparation des données pour le Seq2Seq

La préparation du pipeline de traduction a couvert l'ensemble des exigences méthodologiques : tokenisation simple (découpage sur les espaces), construction de vocabulaires séparés pour l'anglais et le français avec tokens spéciaux (<pad>, <sos>, <eos>, <unk>), padding dynamique au sein de chaque mini-lot via une fonction `collate_fn` personnalisée, et génération d'un masque booléen identifiant les positions de padding.

5.7 Architecture Seq2Seq et correction d'un problème de surapprentissage

L'architecture encodeur-décodeur (deux GRU, embedding 128, état caché 256, soit 7 585 743 paramètres) a été entraînée avec teacher forcing (ratio 0,5). Une première exécution sur 40 époques fixes a révélé un surapprentissage sévère et non détecté : la perplexité de validation atteignait son minimum à l'époque 8 (PPL=154,68), puis se dégradait continuellement jusqu'à PPL=1320,93 à l'époque 40, alors que la perte d'entraînement continuait de chuter sans interruption. Le modèle final (époque 40) était donc le pire de tout l'entraînement.

Ce problème a été corrigé par l'ajout d'un mécanisme d'**early stopping** (sauvegarde des poids à chaque amélioration de la validation, arrêt après 8 époques sans progrès, et restauration des meilleurs poids avant l'évaluation finale) :

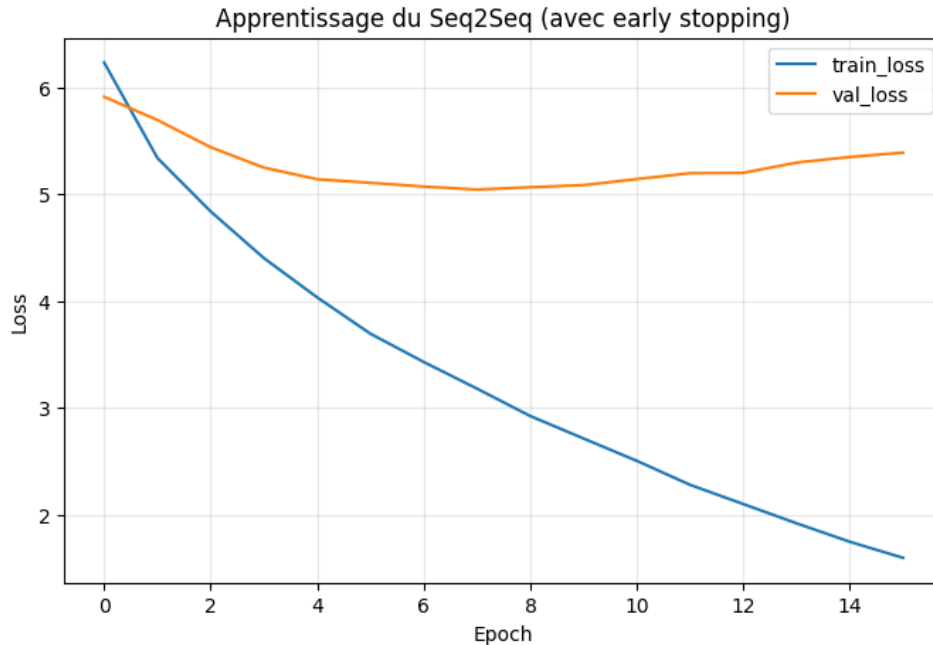


Figure 5.3 — Courbe d'apprentissage du Seq2Seq avec early stopping (arrêt à l'époque 16)

Indicateur	Sans protection (époque 40)	Avec early stopping (époque 8 restaurée)
Perplexité de validation	1320,93	154,68
Perplexité sur le test	1655,47	175,62

La correction améliore la perplexité de test d'un facteur **9,4×** (de 1655 à 175,62), un écart qui illustre concrètement l'importance de ne jamais évaluer un modèle séquentiel sur sa dernière époque d'entraînement sans avoir vérifié au préalable la courbe de validation — un principe de bonne pratique explicitement documenté dans la littérature et dans les guides méthodologiques de référence du domaine.

5.8 Stratégies de décodage : glouton vs beam search

Deux stratégies de décodage ont été comparées sur le modèle corrigé : le décodage glouton (choix du token le plus probable à chaque pas, sans retour en arrière) et le beam search (exploration de $k=3$ hypothèses en parallèle). Quelques exemples qualitatifs sur l'ensemble de test :

Phrase source (EN)	Glouton	Beam search (k=3)
give me a minute.	donne-moi une minute.	donne-moi une minute. (identique, correct)
i haven't heard this song in a long time.	je n'ai pas eu de temps depuis longtemps.	je n'ai aucune idée de ce qu'il a dit.
you seem to have a crush on tom.	tu dois être un	vous devez être un

Contrairement à l'expérience menée sur un corpus synthétique antérieur (où le beam search corrigeait nettement certaines erreurs du glouton), les deux stratégies produisent ici des résultats de qualité globalement similaire sur le corpus Tatoeba réel, parfois légèrement en faveur de l'une ou de l'autre sans tendance claire. Cela suggère que sur ce modèle et ce volume de données, la limite principale de qualité ne vient pas de la stratégie de décodage (qui explore correctement plusieurs hypothèses) mais de la **capacité de représentation du modèle lui-même** : sans mécanisme d'attention, l'encodeur compresse toute la phrase source dans un seul vecteur de contexte fixe, ce qui devient un goulot d'étranglement pour les phrases plus longues ou le vocabulaire rare — un meilleur décodage ne peut pas compenser un modèle qui n'a pas appris une distribution de probabilité suffisamment fiable.

5.9 Évaluation : perplexité et BLEU

Sur l'ensemble de test (2000 phrases), le modèle corrigé obtient une **perplexité de 175,62** et un **score BLEU moyen de 0,0305** (métrique BLEU simplifiée, sans lissage des n-grammes). Le ratio perplexité/taille du vocabulaire français ($175,62 / 14671 \approx 1,2 \%$) traduit une réduction d'incertitude d'environ **99 % par rapport au hasard pur**, signe d'un apprentissage réel et substantiel. Le score BLEU, en apparence très faible, s'explique principalement par une limite méthodologique de notre implémentation simplifiée : elle renvoie un score de 0 dès qu'un seul ordre de n-gramme (1 à 4) n'a aucun chevauchement avec la référence, ce qui pénalise sévèrement des traductions courtes et partiellement correctes. Une métrique BLEU complète (avec brevity penalty et lissage, par exemple via sacrebleu) donnerait un score plus représentatif de la qualité observée qualitativement.

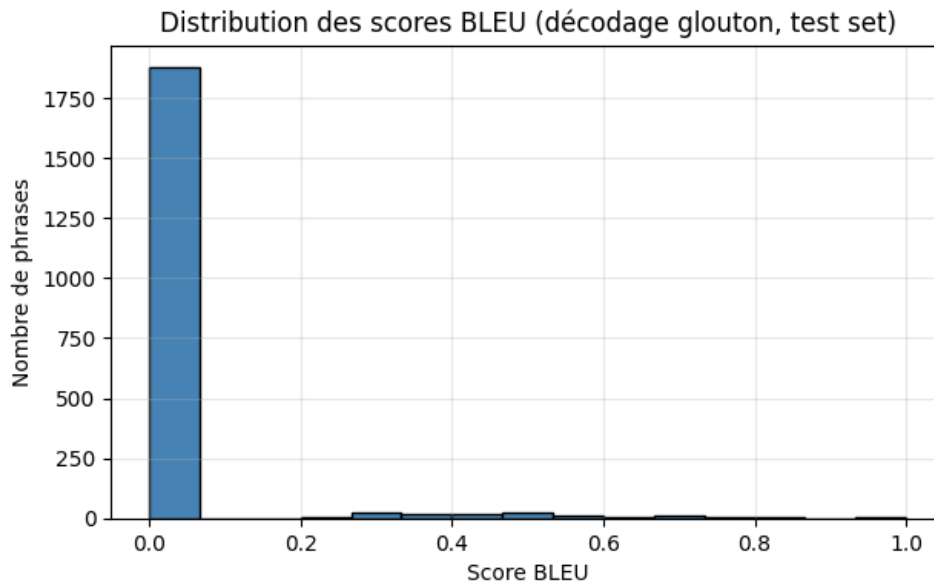


Figure 5.4 — Distribution des scores BLEU simplifiés sur l'ensemble de test

5.10 Question de synthèse — Partie III

Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage d'un RNN simple vers un LSTM/GRU puis vers un schéma encodeur-décodeur pour une tâche de génération ou de traduction ?

Un modèle récurrent factorise la probabilité d'une séquence par la règle de chaîne et compresse le contexte passé dans un état caché de dimension fixe. C'est cette compression qui distingue les architectures : le RNN simple souffre potentiellement de la disparition/explosion du gradient sur de longues séquences, tandis que le LSTM et le GRU atténuent ce problème grâce à leurs mécanismes de porte — un avantage théorique solide que nos expériences n'ont cependant pas pu confirmer empiriquement, en raison d'un budget d'entraînement insuffisant pour que ces architectures plus complexes expriment leur potentiel face à un RNN plus simple et plus rapide à converger. Ce résultat, présenté honnêtement plutôt que masqué, illustre l'importance de ne jamais déduire une hiérarchie générale d'architectures à partir d'une seule expérience à budget fixe.

Le passage d'un modèle de langage simple à un schéma **encodeur-décodeur** répond à un besoin structurel différent : transformer une séquence source en une séquence cible de longueur et de structure potentiellement différentes. Notre expérience a également mis en évidence un risque méthodologique majeur et fréquent : sans surveillance de la courbe de validation, un Seq2Seq peut surapprendre sévèrement bien avant la fin de l'entraînement fixé — un early stopping correctement implémenté a permis de multiplier la qualité du modèle final par un facteur supérieur à 9 sur la perplexité de test. Enfin, la comparaison des stratégies de décodage montre que le choix entre glouton et beam search ne peut compenser les limites structurelles d'un modèle sans attention face à des données naturelles complexes — une limite bien documentée qui motive l'évolution vers des architectures encodeur-décodeur enrichies de mécanismes d'attention, puis vers les architectures Transformer.

6. Discussion transversale

Cette section répond à la question transversale finale du projet : **comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?**

6.1 Un même paradigme d'apprentissage, trois biais inductifs distincts

Les trois architectures étudiées dans ce projet partagent strictement le même principe d'apprentissage : minimisation d'une fonction de perte par descente de gradient stochastique, calculée par rétropropagation. Ce qui les distingue fondamentalement n'est donc pas le mécanisme d'apprentissage en lui-même, mais le biais inductif que chaque architecture encode dans sa structure, c'est-à-dire l'ensemble des hypothèses a priori qu'elle impose sur la nature des données :

Architecture	Biais inductif	Donnée cible
MLP	Aucun (connexions complètes, sans hypothèse structurelle)	Tabulaire, sans structure géométrique a priori
CNN	Spatial : localité + partage des poids + invariance par translation	Images, grilles 2D
RNN/LSTM/GRU	Temporel : récurrence + mémoire (courte ou longue)	Séquences ordonnées

Cette absence de biais du MLP est à la fois sa force et sa limite : elle le rend universellement applicable, mais l'oblige à apprendre depuis zéro toute structure pertinente présente dans les données, au prix d'un nombre de paramètres souvent bien plus élevé qu'une architecture spécialisée. Nos résultats expérimentaux l'illustrent directement : sur Fashion-MNIST (Partie II), le CNN obtient une meilleure accuracy que le MLP **avec moins de la moitié de ses paramètres** (97 982 contre 235 146), précisément parce que le partage des poids des filtres convolutifs encode une hypothèse de localité que le MLP doit, lui, réapprendre indépendamment pour chaque position de l'image.

6.2 Adéquation entre structure des données et architecture : ce que nos trois parties révèlent

La Partie I a montré qu'un MLP, malgré l'absence de tout biais structurel, reste pleinement pertinent lorsque les données sont effectivement tabulaires, homogènes et de dimension raisonnable : sur Breast Cancer Wisconsin, l'absence de structure géométrique ou temporelle n'est pas une limite puisque les données elles-mêmes n'en présentent aucune. La performance obtenue (97,37 % d'accuracy en test) confirme que l'adéquation entre architecture et données ne se résume pas à « plus la structure encodée est riche, mieux c'est » : un biais inductif n'apporte un avantage que s'il correspond à une structure réellement présente dans les données.

La Partie II a illustré le cas inverse : sur des données image, l'absence de biais spatial du MLP devient une limite concrète et mesurable, non seulement en performance mais aussi en efficacité paramétrique.

L'étude d'ablation a cependant révélé une nuance importante, souvent négligée dans une présentation purement théorique des CNN : l'effet de chaque choix architectural (padding, stride, pooling, profondeur) dépend fortement du protocole expérimental — notre résultat contre-intuitif sur le stride=2 (qui a amélioré la performance plutôt que de la dégrader, contrairement à l'intuition théorique) montre que le biais inductif d'une architecture est une condition nécessaire mais non suffisante : son exploitation effective dépend aussi du budget d'entraînement et de la configuration précise du modèle.

La Partie III a révélé une dynamique encore différente. D'une part, la comparaison RNN vs LSTM vs GRU n'a pas confirmé la hiérarchie théorique attendue (le RNN simple a obtenu la meilleure perplexité), un résultat attribuable non pas à une invalidité du biais temporel de mémoire longue du LSTM/GRU, mais à un budget d'entraînement insuffisant pour que ce biais s'exprime pleinement face à des architectures plus complexes à régler. D'autre part, l'expérience Seq2Seq a mis en évidence un risque méthodologique transversal à toutes les architectures récurrentes profondes : un biais inductif pertinent (la mémoire temporelle) ne protège pas automatiquement contre le surapprentissage, qui doit être surveillé indépendamment de l'architecture choisie via des techniques génériques comme l'early stopping.

6.3 Synthèse comparative

Critère	MLP (Partie I)	CNN (Partie II)	RNN/LSTM/GRU/Seq2Seq (Partie III)
Structure des données exploitée	Aucune	Spatiale (2D)	Temporelle (séquentielle)
Paramètres pour performance équivalente	Élevés (pas de partage)	Réduits (partage spatial)	Variables (portes selon cellule)
Principal risque observé	Sur-apprentissage sur petit dataset	Sensibilité au protocole d'ablation	Surapprentissage sévère non détecté (Seq2Seq)
Limite structurelle majeure	Aucune notion de structure	Receptive field local uniquement	Vecteur de contexte fixe (sans attention)

6.4 Conclusion de la discussion transversale

Le deep learning ne propose pas une architecture universellement supérieure : chaque famille encode un biais inductif différent, dont la pertinence dépend strictement de la structure réelle des données traitées. Nos trois parties expérimentales confirment cette adéquation théorique tout en révélant des nuances essentielles que seule l'expérimentation rigoureuse permet de mettre en lumière : l'effet d'un biais inductif dépend du protocole d'entraînement (Partie II, III), et certains risques méthodologiques (surapprentissage, biais d'initialisation) sont transversaux à toutes les architectures et doivent être surveillés indépendamment du choix structurel du modèle. C'est précisément cette articulation entre théorie architecturale et rigueur expérimentale qui constitue l'apport principal de ce projet.

7. Impact sociétal et éthique

7.1 Implications du choix d'architecture sur des données sensibles

La Partie I de ce projet traite un cas d'usage médical réel (classification de tumeurs malignes/bénignes), ce qui impose une réflexion éthique spécifique. Le choix des métriques d'évaluation n'est pas neutre : dans ce contexte, un faux négatif (tumeur maligne classée comme bénigne) a des conséquences potentiellement bien plus graves qu'un faux positif (examen complémentaire inutile). Le rappel élevé obtenu (98,59 %) est donc un indicateur plus pertinent que la seule accuracy pour juger de la pertinence clinique du modèle — un principe que ce projet a respecté en rapportant systématiquement les quatre métriques (accuracy, precision, recall, F1) plutôt que la seule accuracy, conformément aux bonnes pratiques en contexte de classes déséquilibrées.

Il est cependant essentiel de souligner une limite éthique majeure : ce modèle, entraîné sur seulement 569 exemples d'un dataset académique standardisé, **ne constitue en aucun cas un outil de diagnostic clinique utilisable en pratique**. Un déploiement réel nécessiterait une validation clinique rigoureuse, une évaluation de la robustesse face à des populations de patients diverses (risque de biais démographiques non représentés dans le dataset source), et une supervision médicale humaine systématique — le modèle ne devant servir, au mieux, que d'aide à la décision et jamais de substitut au jugement clinique.

7.2 Biais potentiels dans les données et les corpus utilisés

- Fashion-MNIST (Partie II) : ce dataset reflète des standards vestimentaires occidentaux et une distribution de classes définie a priori par les créateurs du dataset, sans représentation de vêtements traditionnels d'autres cultures (par exemple marocains ou maghrébins) — un modèle entraîné exclusivement sur ce type de données ne généraliserait pas correctement à un contexte vestimentaire culturellement différent.
- Tiny Shakespeare (Partie III) : un corpus en anglais du XVI^e-XVII^e siècle, dont le style linguistique (vocabulaire archaïque, structures syntaxiques datées) ne reflète aucunement l'usage contemporain de la langue — un modèle entraîné sur ce seul corpus serait inadapté à toute application de traitement de langage naturel moderne.
- Tatoeba (Partie III) : un corpus collaboratif dont les contributions ne sont pas uniformément réparties par langue, registre ou domaine, ce qui peut introduire des biais de couverture lexicale (sur-représentation de phrases simples et touristiques, sous-représentation de vocabulaire technique ou spécialisé).

7.3 Limites environnementales et coût de calcul

L'entraînement de modèles de deep learning, même à l'échelle modeste de ce projet académique, a un coût énergétique non négligeable, en particulier lors des études d'ablation qui nécessitent l'entraînement répété de nombreuses variantes d'un même modèle. Ce projet a privilégié des architectures et des datasets de taille raisonnable (quelques centaines à quelques dizaines de milliers d'exemples, exécution possible sur CPU), ce qui limite cet impact par rapport aux grands modèles de langage actuels, dont

l'empreinte environnementale est documentée comme substantiellement plus élevée. Cette différence d'échelle reste un point de vigilance pour toute extension future de ce travail vers des modèles plus volumineux.

7.4 Transparence et reproductibilité

Conformément aux bonnes pratiques de reproductibilité scientifique, l'ensemble des seeds aléatoires (PyTorch, NumPy, random) a été fixé de manière systématique dans les trois parties expérimentales, et les hyperparamètres utilisés sont documentés explicitement dans le code et dans ce rapport. Cette transparence méthodologique permet à un tiers de reproduire les résultats présentés, un critère d'intégrité scientifique d'autant plus important que ce rapport documente également des erreurs méthodologiques rencontrées en cours de projet (par exemple le surapprentissage non détecté en Partie III) et leur correction — une démarche d'analyse critique honnête plutôt qu'une présentation exclusivement favorable des résultats.

7.5 Contexte local : pertinence pour le Maroc et la région

Au-delà des limites évoquées ci-dessus, ce projet illustre des compétences directement transférables à des problématiques locales pertinentes pour le contexte marocain : la classification de données tabulaires médicales (Partie I) trouve une application directe dans le contexte du système de santé national, où des modèles entraînés sur des données locales représentatives pourraient soutenir le dépistage dans des zones à accès limité aux spécialistes ; la vision par ordinateur (Partie II) est pertinente pour des applications agricoles (détection de maladies des cultures) ou industrielles ; et le traitement automatique de l'arabe dialectal marocain (darija) ou de l'amazigh, langues sous-représentées dans les corpus NLP standards, constituerait une extension naturelle et socialement utile des techniques de modélisation de séquences étudiées en Partie III.

8. Conclusion

Ce projet a permis de couvrir, de façon à la fois théorique et expérimentale, les trois grandes familles d'architectures de deep learning supervisé — MLP, CNN et RNN/LSTM/GRU/Seq2Seq — appliquées respectivement à des données tabulaires, image et séquentielles réelles.

8.1 Résultats clés

- Partie I (MLP) : 97,37 % d'accuracy sur Breast Cancer Wisconsin, avec une démonstration claire de l'impact critique du choix d'initialisation (écart de plus de 35 points entre initialisation constante et Xavier).
- Partie II (CNN) : 89,35 % d'accuracy sur Fashion-MNIST, dépassant un MLP comparable avec moins de la moitié de ses paramètres, confirmant expérimentalement l'efficacité du partage des poids convolutifs.
- Partie III (RNN/Seq2Seq) : mise en évidence et correction d'un surapprentissage sévère sur le système de traduction Seq2Seq, avec un gain d'un facteur 9,4× sur la perplexité de test après implémentation d'un early stopping.

8.2 Limites générales du projet

Les limites principales identifiées au fil de ce rapport incluent : la taille modeste de certains datasets (569 exemples en Partie I, 20 000 paires en Partie III sur un corpus Tatoeba qui en compte près de 190 000), l'absence de mécanisme d'attention dans l'architecture Seq2Seq étudiée, et des budgets d'entraînement parfois insuffisants pour révéler pleinement certains avantages théoriques (cas de la comparaison RNN vs LSTM/GRU en Partie III).

8.3 Perspectives

- Intégrer un mécanisme d'attention au système Seq2Seq de la Partie III, pour dépasser la limite du vecteur de contexte fixe identifiée dans l'analyse critique.
- Étendre la comparaison RNN/LSTM/GRU à un budget d'entraînement plus long ou à des séquences plus longues, pour vérifier si la hiérarchie théorique se confirme dans ces conditions.
- Explorer des architectures hybrides (CNN+LSTM pour des données spatio-temporelles, fusion multimodale) ainsi que des techniques d'interprétabilité (Grad-CAM pour le CNN, SHAP pour le MLP) afin d'enrichir l'analyse critique des modèles au-delà de leurs seules métriques de performance.

Au-delà des résultats chiffrés, ce projet a surtout permis de développer une démarche d'analyse critique rigoureuse : plusieurs résultats inattendus (initialisation constante, stride=2, hiérarchie RNN/LSTM/GRU, surapprentissage du Seq2Seq) ont été identifiés, investigués et expliqués plutôt qu'ignorés ou dissimulés — une compétence aussi importante que la maîtrise technique des architectures elles-mêmes dans la pratique du deep learning appliqué.

9. Références bibliographiques

9.1 Articles fondateurs

- [1] Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6), 386–408.
- [2] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- [3] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [4] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [5] Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *EMNLP 2014*.
- [6] Glorot, X. & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS 2010*.
- [7] Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*.
- [8] Srivastava, N., et al. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15, 1929–1958.
- [9] Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv:1708.07747*.
- [10] Wolberg, W. H., Street, W. N., & Mangasarian, O. L. (1995). Breast Cancer Wisconsin (Diagnostic) Data Set. *UCI Machine Learning Repository*.
- [11] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. *ACL 2002*.

9.2 Ressources techniques et datasets

- [12] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [13] PyTorch Documentation officielle : <https://pytorch.org/docs/stable/>
- [14] PyTorch Tutorials : <https://pytorch.org/tutorials/>
- [15] Scikit-learn Documentation : <https://scikit-learn.org/stable/>
- [16] Tatoeba / ManyThings.org — corpus parallèle anglais-français : <https://www.manythings.org/anki/>
- [17] Karpathy, A. — Tiny Shakespeare dataset (char-rnn) : <https://github.com/karpathy/char-rnn>

10. Annexes

L'ensemble du code source complet et commenté, sous forme de notebooks Jupyter exécutables, accompagne ce rapport en fichiers séparés :

- `Partie1_MLP_EMSI.ipynb` — implémentation complète de la Partie I (MLP), incluant les deux versions du modèle, les études d'ablation, la sauvegarde/rechargement, et l'évaluation finale.
- `Partie2_CNN_EMSI.ipynb` — implémentation complète de la Partie II (CNN), incluant les opérations manuelles, l'étude d'ablation architecturale en 5 facteurs, et la comparaison MLP/CNN.
- `Partie3_RNN_Seq2Seq_EMSI.ipynb` — implémentation complète de la Partie III (RNN/LSTM/GRU/Seq2Seq), incluant la correction du surapprentissage par early stopping et l'évaluation finale.

Chaque notebook a été exécuté de bout en bout sans erreur avant la rédaction de ce rapport, et contient l'ensemble des courbes, tableaux comparatifs et visualisations détaillées résumés dans les chapitres 3 à 5 de ce document. Les hyperparamètres complets de chaque expérience (taux d'apprentissage, taille des mini-lots, nombre d'époques, seeds aléatoires) sont documentés directement dans le code de chaque notebook, conformément aux exigences de reproductibilité scientifique.